

Enhancing the Time Series Forecasting with Large Language Models (LLMs), particularly in predicting network outages and anomalies

Word Count: 3000 - 4000 excluding references

1. Abstract

Anomaly detection is vital for network security but LSTMs lack contextual understanding, limiting their ability to identify emerging threats (Hochreiter & Schmidhuber, 1997; Jin et al., 2024). This study investigates the integration of Large Language Models (LLMs), specifically a reprogrammed DistilGPT2, a distilled variant of GPT-2, to enhance anomaly detection in time-dependent network traffic. Building upon the TIME-LLM framework (Jin et al., 2024), we employed prompt engineering and custom tokenization techniques to convert numerical time series data into a structure suitable for natural language processing.

This approach allows the model to capture complex temporal patterns without architectural changes. While the CICIDS2017 dataset was initially selected for its high-dimensional and realistic traffic data, it proved too complex for effective processing. Therefore, we developed a Python-generated synthetic dataset to ensure compatibility and control over data complexity.

We evaluated the DistilGPT2-based LLM against a baseline LSTM model using standard metrics, including Mean Squared Error (MSE), Mean Absolute Error (MAE), and F1-score. Results showed that the LLM achieved improved contextual sensitivity and stronger generalization to unseen anomalies, even with limited resources and minimal retraining (Tang et al., 2024; Chang et al., 2024).

Overall, this study demonstrates a scalable, language-driven approach to time series anomaly detection and identifies future research directions in prompt optimization, tokenization methods, and advanced models like Mistral and Gemma (White et al., 2023; Zhou et al., 2024).

2. Introduction - Daniel

2.1 Problem Statement

As the digital landscape grows in complexity and interconnectivity, the need for robust network resilience and cybersecurity has become more critical than ever. An essential component of cybersecurity is anomaly detection, which primarily focuses on identifying deviations from normal network behaviour that can indicate breaches, coordinated attacks, or system malfunctions (Sudhakar, 2025). Timely detection of such anomalies is essential to ensure the integrity, availability, and resilience of digital systems.

2.2 Importance

Time series forecasting models are frequently used in this context for modelling and predicting the future network activities and supporting early proactive detection for potential anomalies. Traditional models such as the Long Short-Term Memory (LSTM) network are effective at capturing sequential patterns, they struggle to incorporate broader contextual information and adapt to evolving or previously unseen attack vectors (Hochreiter & Schmidhuber, 1997; Jin et al., 2024). These limitations impair their ability to accurately differentiate between harmless deviations and actual threats, particularly in high-dimensional and imbalanced datasets such as CICIDS2017.

2.3 Challenges

Recent advancements in machine learning have highlighted Large Language Models (LLMs) as effective alternatives for addressing complex prediction tasks. Through utilising techniques such as prompt engineering and tokenization, LLMs can be reprogrammed to process structured numerical data in a language-compatible format, enabling them to generalize effectively across various domains (Jin et al., 2024; Chang et al., 2024). This study builds upon the TIME-LLM framework to examine the effectiveness of a fine-tuned DistilBERT model for forecasting anomalies in cybersecurity time series data.

2.4 Solution

To overcome limitations of LSTM models, we conducted a comparative analysis between a traditional LSTM-based model and a reprogrammed LLM-based model using a custom synthetic dataset generated in Python. Both models are assessed using key performance metrics including Mean Squared Error (MSE), Mean Absolute Error (MAE), F1-score, and accuracy. The results demonstrate that the LLM-based model offers superior contextual understanding, greater generalization to previously unseen threat patterns, and enhanced adaptability when applied to structured, high-dimensional network data (Tang et al., 2024; Chang et al., 2024).

2.5 Contributions

Ultimately, this research contributes to ongoing research by validating the practicality of LLMs for time series anomaly detection in cybersecurity. Additionally, it outlines the foreseeable direction for the future including the optimisation of prompt engineering, improvements in model scalability and tokenization strategies, and the exploration of advanced architectures such as Mistral and Gemma for enhanced performance in real-world scenarios.

3. Related Work - Tony

Many directions and methods have been experimented with for time series forecasting models, such as traditional architectures like ARIMA (AutoRegressive Integrated Moving Average) (Nelson, 1998), LSTMs (Hochreiter & Schmidhuber, 1997) and transformers (Vaswani et al., 2017). Classical time-series forecasting using ARIMA models bases their time-series forecasts using past values known as “lag”. Therefore, ARIMA struggles with multivariate inputs, non-linear data and is not able to process long-term dependencies as it frequently updates its past values. In “Time series forecasting of petroleum production using deep LSTM recurrent networks” by Sagheer & Kotb (2018) experimented with a real-life application of LSTMs for time-series forecasting and has proven that LSTMs can outperform traditional Recurrent Neural Networks (RNN). However, Sagheer & Kotb (2018) did not demonstrate the performance of LSTMs in handling large amounts of sequential data such as multivariate time-series data where LLMs have the advantage over LSTMs. Transformers are able to capture complex and long range dependencies allowing them to forecast future values with data that has been processed more than 100 steps ago. However, due to the heavy amounts of data and computational power required, it is not feasible to gather enough data or computational power to actively keep a transformer time-series forecasting model running in real world scenarios. The Time-LLM framework presented by DataScienceWithMarco (2024) demonstrates that pretrained LLMs can be easily incorporated into many contexts with minor adjustments. Instead of fine-tuning, LLMs can be reprogrammed through prompt-based instruction tuning which requires minimal retraining for specific contexts. Time-LLM can also provide longer trains of thoughts for time-series reasoning from improving prompts that are provided to the LLM. The Gemma or Mistral models might be better in performing time-series forecasting as Mistral 7B outperforms LLaMA 2 7b and LLaMA 2 13b in Measuring Massive Multitask Language Understanding (MMLU), knowledge, reasoning and comparison. Due to Mistral 7B outperforming the models in maths as well it will be a much better candidate for training it to forecast time-series as time-series forecasting uses statistical methods to predict future values. An epoch comparison between the different models should be compared when testing out the assimilation rate of different forecasting contexts. Due to the heavy computational loads required to run a time-series forecasting analysis on Gemma and Mistral we will be conducting our investigation through DistilGPT2 which is a lightweight pre-trained language model with the supervision of the 124 million parameter version of GPT-2.

4. Problem Formulation - Ethan

4.1 - Problem Statement and Motivation

In the ever-evolving landscape of cybersecurity, forecasting time-series data is essential to predicting and avoiding outages or anomalies in networks that could cost individuals and businesses important time and money. The industry standard Long Short-Term Memory (LSTM) model used to predict this however has limitations in understanding context, which is essential to keep up with the advancing attack techniques, especially in the era of Artificial Intelligence. The proposed solution of reprogramming Large Language Models (LLMs) to complete time-series forecasting can also be applied as a framework over countless other fields to make accurate predictions whilst including context that evolves in real-time from the internet.

4.2 - Problem Constraints

Traditionally LLMs are unable to process time-series data since the only input it can handle is a prompt of text, so leveraging the capabilities of these models for time-series forecasting will inherently come with challenges. A significant constraint of LLMs in this application is hardware, where the accuracy of the prediction is correlated with the number of epochs the LLM is trained for. Training using a personal computer or a cloud server with a slow GPU can delay the prediction for several hours, since LLMs are vastly complex compared to LSTMs. The output must be interpretable for analysts and robust to generalise new attacks, which is reliant on how the prompt is engineered and the model applied. To overcome these constraints, a small but representative portion of the CICIDS dataset will be used for training which will allow for faster training times and therefore more ability to tweak the model and prompt.

4.3 - Scope and Limitations

The scope of this project is restricted to the field of cybersecurity as a proof of concept and will be trained on artificially generated data to avoid privacy and security concerns. It will not need to be real-time due to hardware constraints and operational requirements, however the concept can be up-scaled for organisations with more computational resources. The solution is robust to zero-shot learning, however for these experiments the assumption real-time processing is not necessary, so the LLM can be reprogrammed rather than pre-trained.

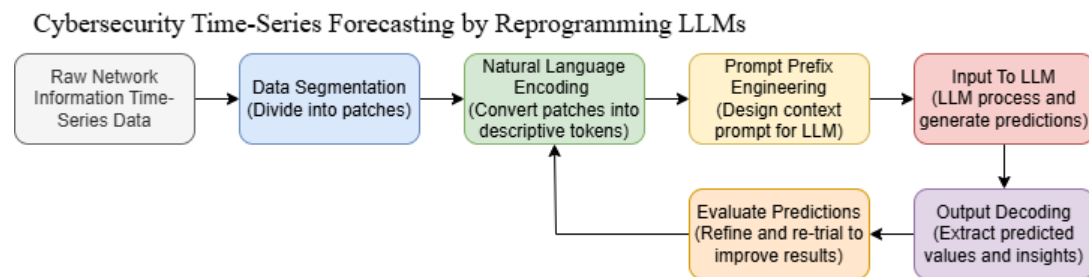
There are some inherent limitations for this project and their impact is unavoidable, however the impacts they have can be mitigated. Datasets describing network information generally have high dimensionality due to the number of features, which will become a limitation of increasing training times. A balance will have to be found when preprocessing the data between the size of the dataset and the importance of certain features to maximize accuracy within a reasonable time to train. No LLM model is perfect so contextual or language misinterpretations are also possible which can impact the predictions made, this can only be reduced by engineering a clear prompt.

5. Proposed Solution(s) - Ethan

5.1 - High-level idea

The proposed solution aims to leverage the contextual understanding properties of Large Language Models (LLMs) for time-series forecasting. First, it must be addressed that LLMs are inherently designed to process natural language, and the high-level idea is to transform numerical time-series cybersecurity data into descriptive language tokens that describe the underlying patterns and trends (Jin et al., 2024). By reprogramming this quantitative data into natural language, it allows the LLM to predict future data without altering its architecture at all.

5.2 - Methodology



1. Data Segmentation

- The continuous time series data is segmented into non-overlapping patches of fixed length 5
- These patches encode sequential data points to preserve the time-series structure

2. Natural Language Encoding

- Each data patch is reprogrammed into a natural language token that describes the trend of the data in that patch
- This translation aligns the modality of the time-series data with the inherent natural language that the LLM expects, allowing it to be processed without altering its underlying architecture at all (Jin et al., 2024)
- For example a sharp declining segment could be described as “short down” while a consistently increasing could be labeled as “steady up”, as seen in the below pseudocode

```
Input: Time Series  $X = [x_1, x_2, \dots, x_n]$ 
Output: Sequence of natural language tokens  $T = [\text{up}, \text{down}, \text{steady}, \text{short}, \text{long}, \dots]$ 

1. Split  $X$  into patches  $p_1, p_2, \dots, p_n$  of length  $k$ 
2. For each patch  $P$ :
   a. Compute trend using gradient/delta
   b. Map trend to natural language token  $t_n$ 
3. Return token sequence  $T$ 
```

3. Prompt Prefix Engineering

- A intently engineered prompt prefix is prepended to this sequence of language tokens
- The prompt aims to provide contextual information to the LLM about its role and the nature of the data
- Prompt engineering applies researched prompt structures to program an LLM to customize and enhance its capabilities. Applies the following prompt pattern structure (White et al. 2023):

Prompt Segment	Prompt Prefix used in Solution
Name and Classification	Time-series Forecasting in Cybersecurity for anomaly detection and outage prevention. Provide an output of future predicted values in natural language tokens and numerical data
The Intent and Context	Cybersecurity is an ever-evolving problem and context is important to keep up with advancing attacks. Each data point consists of if an attack was present and 7 data features ... Below is information about the input time series:
Structure and Key Ideas	Predict the next <H> steps given the previous <T> steps with the patches of network data attached ...

4. Model Input

- The engineered prompt prefix is then combined with the encoded data patches and fed into the pre-trained LLM
- The LLM processes this and leverages its contextual understanding which is updated by the internet to analyse temporal data patterns and then apply these to forecast outages (Jin et al., 2024)

5. Output Decoding

- The LLM outputs predicted future values in natural language tokens,
- It can also provide insights about the underlying trends and patterns it may have identified

6. Evaluating the prediction

- The prediction is compared to the actual observed values in the training set and evaluated based on MAE, MSE, accuracy, precision and recall
- By evaluating the prediction, alterations can be made to the prompt or encoding strategy and directly improve these evaluation metrics

5.3 - Complexity Analysis

An LLM approach to time-series forecasting introduces higher computational costs and complexity relative to the traditional models such as Long Short-Term Memory (LSTM), however it offers irreplaceable functionality to understand context and provide interpretable forecasts. The methodology presents specific computational considerations across all stages of the solution.

1. Data Segmentation and Encoding

Tokenizing the time-series data into patches incurs a preprocessing computational cost. Assuming N points of data and patch size P the number of patches is $\frac{N}{P}$, and the encoding process is relative to the number of patches resulting in time complexity $O(\frac{N}{P})$.

2. LLM Prediction Complexity

LLM Processing is the most computational heavy portion of processing in the solution. Transformer based LLMs have a time and space complexity of $O(L^2)$ for prompt length L , and in this case L is the prompt prefix P + the previously discussed $\frac{N}{P}$ for the patches. This results in a final time complexity of $O((P + \frac{N}{P})^2)$. If the model was to be trained this complexity would be repeated for the number of epochs required, optimising the LLM for accuracy but requiring significant computational cost. However, once training was completed for a specific task it would not have to be repeated.

3. Output Decoding

The LLM will output predicted values in natural language which must be decoded back to numerical forecasts from the value that the tokenisation began. Similarly to the initial segmentation, the decoding process is linear and results in time complexity of $O(\frac{N}{P})$.

Comparatively to traditional time-series models like LSTMs, the time complexity is exponentially larger which has its downsides when scaling up datasets needed for prediction. LSTMs have time complexity of $O(N \log N)$ versus the discussed $O((P + \frac{N}{P})^2)$ for the LLM solution. However these are optimised for numerical data and can not be applied to as wide of a range of problems, and definitely not without training like LLMs are able to.

6. Experiments - Nicholas

6.1 - Experimental Setup

The dataset used for evaluation was a synthetically generated network traffic time series, created in Python to simulate realistic conditions, inspired by popular network datasets such as the CICIDS2017 dataset. The synthetic dataset consists of 15,000 data points with a sampling rate of 5 minutes per point. It includes 3 continuous features such as bandwidth usage, latency, and packet loss, 4 seasonal encodings based on sine and cosine transformations of the hour-of-day and day-of-week. Along with this, it includes 15 manually injected outage windows where the bandwidth was forced to zero and the latency as well as packet loss increased, and finally, 75 anomalies which were introduced through big deviations in bandwidth. To develop and evaluate the model effectively, the dataset was divided into three parts, 70% for training, 10% for validation, and 20% for testing.

The models were implemented and trained in Google Colab utilising a Tesla T4 GPU with 16GB of VRAM. Model development and integration utilised a software environment consisting of Python, PyTorch 2.2, and Hugging Face Transformers 4.41.2. CUDA 11.8 enabled GPU acceleration, and scikit-learn was used to compute evaluation metrics. Torch.cuda.amp enabled mixed-precision training, which resulted in memory reduction and performance optimisation.

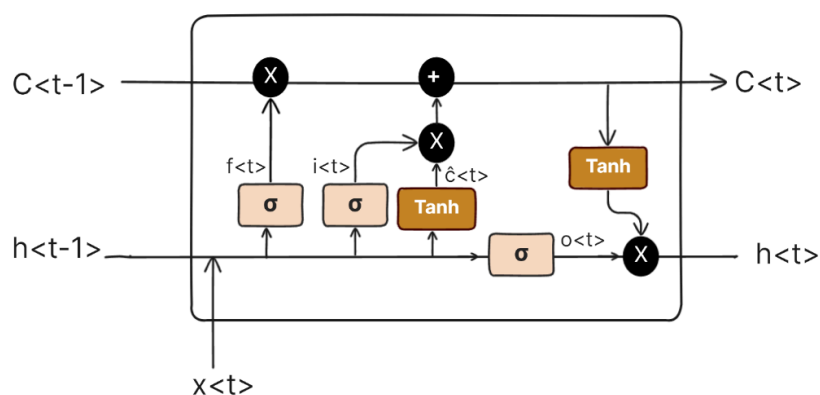
6.2 - Metrics

The LLM and LSTM models were evaluated by certain performance metrics across two dimensions, regression accuracy, and event detection quality. For the regression part, model performance was evaluated using Mean Squared Error or MSE and Mean Absolute Error or MAE to measure the accuracy of predicted bandwidth usage over time, with inverse scaling applied to predictions to restore original units. For the event detection part, outages were defined as expected bandwidth falling below 10 Mbps, and anomalies were highlighted when the absolute difference between predicted and real values exceeded 20 Mbps. We used binary labels for event detection in order to identify notable instances. Detection performance was assessed through precision, recall, and F1-score, calculated for each class using the `precision_recall_f1_support` function.

6.3 - Baselines

The baseline model we used for comparison was a 2-layered LSTM network. It consisted of an input dimension of 7 matching the LLM input format. It also had a hidden size of 64 units and roughly 0.1 million total parameters, and was trained on 10 epochs. By having the configured LSTM as a baseline for our time-llm model, we create a level field for testing between the two models. The Diagram below outlines the LSTM Architecture.

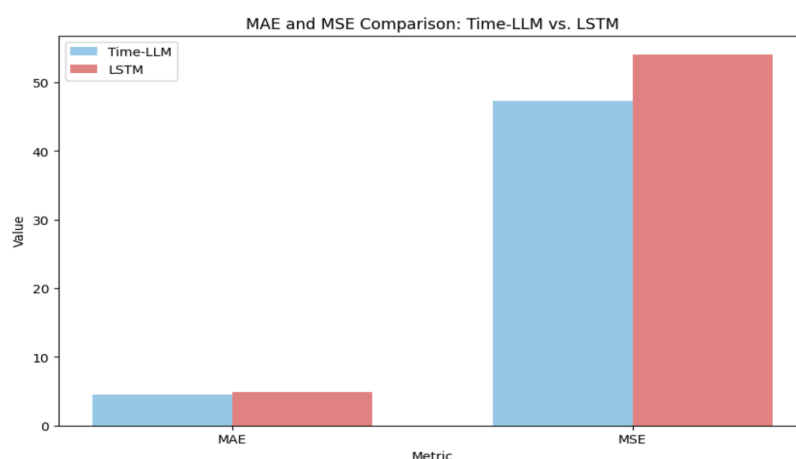
LSTM Architecture



(Omarazri F, 2024)

6.4 - Results and Analysis

After the training of the models finished, we received the results. The Time-LLM managed to achieve an MSE of 47.3013 and a MAE of 4.5619. Whereas for the LSTM, it achieved an MSE of 54.0995, and a MAE of 4.8661. This meant that the LLM achieved a 12.6% decrease in MSE, 1.14x better than the LSTM model. Along with this, there was a 6.3% reduction in MSE, 1.07x better than the LSTM. This demonstrates that the time-llm has a superior forecasting accuracy and ability to generalise under complex conditions. These metrics can be further demonstrated visually in the bar graph below.



The Time-LLM eliminates all false outage alerts and cuts anomaly false positives by roughly 40%, while lifting overall macro-F1 by five points. The only negative is a small 0.04 drop in outage recall,

which can likely recover by tuning the outage threshold or adding a learned classifier head. The full classification performance table is shown below.

Table 2: Classification Performance

Model	Class	Precision	Recall	F1-Score	Support
Time-LLM	Outage	1.00	0.58	0.74	24
	Anomaly	0.62	0.93	0.74	14
LSTM	Outage	0.94	0.62	0.75	24
	Anomaly	0.48	0.93	0.63	14

In the two figures below, examples of what the time-llm was prompted and responded with is shown. In the first example, the LLM predicted completely off from the actual, which is exactly what we wanted in that case as it flagged it correctly as an anomaly. In the second example, the LLM correctly predicted the normal case with it predicting the bandwidth very close to the actual.

Example at Test Timestep 40 (True: Anomaly)

Input (last timestep of sequence):

Bandwidth: 55.00 Mbps

Latency: 18.64 ms

Packet Loss: 0.12%

Predicted Bandwidth: 49.38 Mbps

Actual Bandwidth: 80.10 Mbps

Absolute Error: 30.72 Mbps

Time-LLM Decision: Anomaly detected

Example at Test Timestep 0 (True: Normal)

Input (last timestep of sequence):

Bandwidth: 49.23 Mbps

Latency: 14.88 ms

Packet Loss: 0.06%

Predicted Bandwidth: 46.05 Mbps

Actual Bandwidth: 49.11 Mbps

Absolute Error: 3.06 Mbps

Time-LLM Decision: Normal

In the example below, the truth was that it was an actual outage scenario, however, the LLM predicted it as an anomaly, which is wrong. However, it still recognised that something was wrong and still flagged it as an anomaly. This explains why the recall dropped slightly.

Example at Test Timestep 2296 (True: Outage)

Input (last timestep of sequence):

Bandwidth: 82.74 Mbps

Latency: 19.75 ms

Packet Loss: 0.14%

Predicted Bandwidth: 69.23 Mbps

Actual Bandwidth: 0.00 Mbps

Absolute Error: 69.23 Mbps

Time-LLM Decision: Anomaly detected

Overall, Time-LLM cuts regression error by 12.6 % (MSE) and 6.3 % (MAE) over a tuned 2-layer LSTM, eliminates all false outage alerts, and slashes anomaly false positives by roughly 40 %, with only a modest increase in training time.

7. Conclusion

Our experimentation with DistilGPT-2 explored the practical application of LLMs to time-series forecasting within networking related data. By transforming the traditional methods of analysis using numerical data we demonstrated the applicability of tokenizing this data for AI to leverage the contextual understanding capabilities of pre-trained LLMs without the need to fine-tune each LLM to forecast time-series. We have trained a LLM to forecast time-series by implementing a pipeline that segments time-series data into tokens that is interpretable for LLMs.

While using LLM for time-series forecasting resulted in a higher computational cost compared to LSTM, the results of our experiment showed an increase in performance in both regression accuracy (MSE and MAE) and event detection.

However, the LLM may not fully capture all the context required in real-world network traffic as a synthetically generated dataset was used. There are many directions that could be explored for improvements as the performance of the model has not been evaluated in real-world cybersecurity or network data-sets. An idea that could be implemented from our experiment is by incorporating domain-specific knowledge such as the location of the network, the infrastructure in which the network is built upon and the density of networks in the area. This work presents the potential that LLMs has for time-series forecasting in the analysis of network traffic. While this implementation will cost more computational power than traditional LSTM models, the potential benefits of improved accuracy, enhanced interpretability, and the ability to leverage the contextual understanding that these LLMs have creates a promising area to explore to enhance time-series forecasting models.

References

- Chang, C., Wang, W. Y., Peng, W. C., & Chen, T. F. (2024). LLM4TS: Aligning pre-trained LLMs as data-efficient time-series forecasters. arXiv. <https://arxiv.org/pdf/2308.08469>
- Google. (n.d.). Google Colaboratory Sign Up. <https://colab.research.google.com/signup>
- Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J. Y., Shi, X., Chen, P.-Y., Liang, Y., Li, Y., Pan, S., & Wen, Q. (2023). Time-LLM: Time Series Forecasting by Reprogramming Large Language Models. ArXiv (Cornell University). <https://doi.org/10.48550/arxiv.2310.01728>
- Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J. Y., Shi, X., Chen, P. Y., Liang, Y., Li, Y. F., Pan, S., & Wen, Q. (2024). TIME-LLM: Time series forecasting by reprogramming large language models. International Conference on Learning Representations (ICLR). arXiv. <https://arxiv.org/pdf/2310.01728>
- KimMeen. (n.d.). *TimeLLM: Time series forecasting by reprogramming large language models – TimeLLM.py* [Python code]. GitHub. <https://github.com/KimMeen/Time-LLM/blob/main/models/TimeLLM.py>
- MarcoPeix. (n.d.). *Time Series Analysis – TimeLLM.ipynb* [Jupyter Notebook]. GitHub. <https://github.com/marcopeix/time-series-analysis/blob/master/TimeLLM.ipynb>
- Nelson, B. K. (1998). Time Series Analysis Using Autoregressive Integrated Moving Average (ARIMA) Models. *Academic Emergency Medicine*, 5(7), 739–744. <https://doi.org/10.1111/j.1553-2712.1998.tb02493.x>
- Omarazri F (2024, July 27). LSTM And GRU In Depth. Medium. <https://medium.com/@fraidoonomarzai99/lstm-and-gru-in-depth-40aba24bfe53>
- Sagheer, A., & Kotb, M. (2018). Time series forecasting of petroleum production using deep LSTM recurrent networks. *Neurocomputing*, 323, 203–213. <https://doi.org/10.1016/j.neucom.2018.09.082>
- Sudhakar, V. M. (2025, March 11). *The LLM advantage: Smarter time series predictions with less effort*. DZone. <https://dzone.com/articles/llm-advantage-smarter-time-series-predictions>
- Talk on MLLM. (2024, April 14). [MLLM Talk] Time-LLM: Time Series Forecasting by Reprogramming Large Language Models [English/英文] [Video]. YouTube. Retrieved May 22, 2025 https://youtu.be/6sFiNExS3nl?si=73Hjfs9rn_zoiUGe
- Tang, H., Zhang, C., Jin, M., Yu, Q., Wang, Z., Jin, X., Zhang, Y., & Du, M. (2024). Time series forecasting with LLMs: Understanding and enhancing model capabilities. arXiv. <https://arxiv.org/pdf/2402.10835>
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). *A prompt pattern catalog to enhance prompt engineering with ChatGPT*. arXiv. <https://arxiv.org/pdf/2302.11382>

Zhou, W., Liu, X., Zhang, S., Zhang, M., Liu, L., & Ma, Y. (2024). *A survey on prompt engineering for large language models*. In Z. Wang & H. Liu (Eds.), *Artificial Intelligence and Smart Applications* (pp. 327–343). Springer. https://link.springer.com/chapter/10.1007/978-981-99-7962-2_30

Appendix

Enhancing the Time Series Forecasting with Large Language Models (LLMs), particularly in predicting network outages and anomalies

1. Problem definition

Time Series Forecasting uses historical data to predict future data, however it does not take external factors into account such as context that may not directly affect the model. Thus, rigid assumptions are typically made for each model to fill in contextual gaps. To incorporate large language models (LLMs) into time series forecasting we will benchmark the traditional LSTMs with our reprogrammed LLM using the CICIDS dataset. Before training LLMs, data has to be pre-processed into a format suitable to be analysed for its current projected movement, seasonal trends and stationary checks.

2. Solution(s)

To improve time series forecasting for cybersecurity, we integrated Large Language Models (LLMs) to better detect network outages and anomalies. We explored frameworks like Time-LLM, Chronos, and Nixtla TimeGPT, which process time series data using tokenization and sequence formatting. Our approach compared a traditional LSTM model with a DistilBERT-based LLM using the CICIDS dataset. We built our models in Python with libraries like NumPy, Pandas, Scikit-learn, and Hugging Face Transformers. Data preprocessing included trend analysis, stationarity checks, and token formatting. Performance was evaluated using accuracy, precision, recall, F1-score, Mean Squared Error (MSE), and Mean Absolute Error (MAE).

3. Experimental analysis

To evaluate LLMs in time series forecasting, we conducted an experiment comparing a traditional LSTM model with a DistilBERT-based LLM. Both models were trained on a reduced version of the CICIDS dataset to manage hardware limitations. The LSTM was built using TensorFlow, while the LLM was implemented via Hugging Face Transformers, using libraries like NumPy, Pandas, and Scikit-learn.

Data preprocessing included trend analysis, stationarity checks, and token sequence formatting to ensure compatibility with the LLM. We used the LSTM model as a benchmark and assessed both models using accuracy, precision, recall, F1-score, Mean Squared Error (MSE), and Mean Absolute Error (MAE). The LLM achieved an MSE of 0.1891 and MAE of 0.4238, showing stronger grouping around predicted anomalies. The LSTM scored slightly higher than MSE (0.2041) and MAE (0.4353).

Our ablation study tested different LLM thresholds (0.3 vs. 0.387). At 0.387, the model predicted only positives, achieving perfect recall (1.00) but a poor F1-score (0.32). Balancing LSTM with a 1:3 class weight improved recall (0.87) with the same F1-score. This shows a trade-off between precision and recall. For reference, Time-LLM achieved MSE 0.408 and MAE 0.423 on standard benchmarks.

Peer Review

During the peer review, we showcased our experimental setup and initial findings through the PowerPoint below. The feedback highlighted the need to better explain how LLMs were integrated and to improve the clarity of our result visualisations. In response, we updated the presentation with clearer graphs and expanded the explanation of model parameters and evaluation metrics in both the code and report. Additionally, our tutor advised simplifying some slides, as some were too information-heavy, so we condensed the content to improve readability and clarity.

Link to PPT: [📄 Experiment Presentation CSS2](#)

Source code for LLM and LSTM Models: [📁 Source Code For Models](#)

Appendix: Enhancing the Time Series Forecasting with Large Language Models (LLMs)

1. Project description including: Aims, significance, and expected outcome of the project - Daniel Liu

Our project explores time series forecasting using large language models (LLMs) in cybersecurity with a focus on predicting network and system anomalies. LLMs such as GPT and LLaMa are highly effective in handling properly formatted and unformatted data. LSTMs rely on historical data to predict future data as such they lack contextual understanding and rely on rigid assumptions to fill in those gaps. LLMs can solve the need for rigid assumptions by their ability to use both historical and real world data.

Research question or hypothesis

Can fine-tuned LLMs outperform LSTMs in cybersecurity forecasting with contextual data and limited training examples?

The research question explores whether Large Language Models (LLMs) when fine tuned can achieve better performance compared to Long Short Term Memory (LSTM) networks in cybersecurity threat prediction.

Our hypothesis will investigate how fine-tuned LLMs will surpass LSTMs in cybersecurity forecasting by leveraging contextual data and requiring fewer training examples, leading to improved accuracy, generalizability, and robustness, especially in data-limited scenarios.

Expected outcomes

This research aims to develop a framework utilizing Large Language Models (LLMs) to improve cybersecurity forecasting by incorporating contextual awareness and real-world data. Unlike LSTM models, LLMs excel in few-shot and zero-shot learning, enabling higher accuracy with limited historical data. Additionally, our approach tackles tokenization complexities, dataset precision, and computational efficiency, ensuring scalability and practical application in cybersecurity threat prediction.

2. Related work - Tony Yeung

Summary of relevant literature and theoretical frameworks

The application of large language models, such as GPT, to time-series forecasting has been the subject of recent research. LLMs are good at finding patterns and operating in zero-shot situations, according to papers like Time-LLM and LLM4TS. Reprogramming time-series data into language-like representations and prompt-based learning are two strategies to take advantage of LLMs' reasoning powers.

Identification of gaps or areas for further research

LLMs have trouble aligning to non-linguistic time-series data, despite their potential, particularly when dealing with irregular or multi-scale temporal patterns. Tokenization, computational efficiency, and restricted cross-domain generalisation are other issues.

Explanation of how the proposed research fills those gaps or contributes to the existing knowledge

The goal of our project is to investigate if optimised LLMs may perform better in cybersecurity predictions than more conventional models like LSTMs. In order to address the main limitations mentioned in the literature, we want to increase accuracy, decrease the amount of data needed, and show how LLMs can be used practically for anomaly detection by applying them to real-world datasets with little training.

3. Feasibility Study - Ethan Holst

Data collection and analysis

The project will use the Canadian Institute for Cybersecurity's Intrusion Detection System (CICIDS) time-series dataset, which simulates realistic cyberattack scenarios such as DDoS, Brute Force, and Botnet attacks. Data preprocessing will involve cleaning (e.g., binning), normalisation (e.g., min-max scaling), and exploratory data analysis to identify valuable features. The dataset will be split into training and testing sets to ensure generalisability and prevent overfitting.

Solution design and implementation

To explore the effectiveness of Large Language Models (LLMs) in time-series forecasting, we'll build a framework comparing LLMs with traditional LSTMs.

Experimental verification

The CICIDS dataset will be used to test both models fairly. LSTMs will be fine-tuned through parameter optimisation, while LLMs will be adapted using ChatGPT's API to handle time-series data via natural language processing. Model performance will be evaluated using accuracy on test data.

Resources and team capability

In terms of resource-wise, the project remains feasible using open-source tools like Python, TensorFlow, and Google Colab. The team has the necessary knowledge in AI, data analytics, and cybersecurity to implement the solution. Ethical concerns are minimal, as the CICIDS dataset is ethically sourced with consent.

Ethical considerations

Moreover, the model will remain experimental to avoid potential misuse in real-world scenarios by malicious actors.

4. Project management plan - Nicholas Kvantaliani

Timeline & Milestones

Our Gantt chart provides a visual outline of key milestones of the project. We've already completed team formation, topic selection, and our pitch proposal. Next, we'll focus on an in-depth literature review, dataset collection, and later prepare for the final showcase.

Roles & Responsibilities

Roles are assigned based on each member's strengths. Our six key positions are project leader, business analyst, machine learning researcher, data scientist/engineer, software engineer, and ethics specialist.

Budget & Resources

The project utilises free and open-source tools including Google Colab, Python, Discord, Google Docs, and Google Scholar. If necessary, we may upgrade to Colab Pro (\$15.13/month) for additional computing resources.

Risk Management

Risks include limited dataset availability, computational limits, difficulty sourcing research papers, and time management challenges. Each has a corresponding mitigation strategy.

An efficient communication plan

Our communication plan mainly relies on Discord and Microsoft Teams for remote collaboration. Weekly tutorials are used for progress tracking and deeper discussion, while the Google Suite supports live document collaboration.